

# *MetaMo:*

## *A Robust Motivational Framework for Open-Ended AGI*

### *1 Introduction*

MetaMo as a unified formal framework for AGI motivation systems. It combines three areas of math, category theory, functional analysis, and topology, to support open-ended agents that can change themselves and evolve their own goals and drives over time. The authors mention that a sequel paper (the one you already documented) shows how MetaMo maps onto concrete systems like OpenPsi and MAGUS.

The core of MetaMo is one combined operator  $F$  that does appraisal first and then decision. The appraisal part has comonad structure and the decision part has monad structure, so together they form a pseudo-bimonad. This is what gives a clean "feel vs choose" pipeline. On top of this, MetaMo enforces a contractive update law that keeps motivational states inside a safe region, and it assumes a tubular topology, which basically guarantees that any reachable target can be reached through a thick, feasible path made of small incremental steps.

From this foundation, the paper pulls out five meta-motivational design principles:

- I. ***Modular Appraisal-Decision Interface:*** keep mood updates and goal selection separate, but allow just enough feedback so that swapping their order only changes things slightly.
- II. ***Reciprocal State Simulation:*** share precise state-translation maps so agents can step into each other's motivational frames, enabling smooth hand-off and deep empathy.
- III. ***Parallel Motivational Compositionality:*** run several motivation subsystems (like exploration, ethics, service) in parallel and merge their outputs with small coherence corrections.
- IV. ***Homeostatic Drive Stability:*** apply damping so small disturbances fade out, and tighten control near the boundaries.
- V. ***Incremental Objective Embodiment:*** blend part of the way toward preferred states each cycle, so the agent gradually converges into a feasible ideal region without overshooting, destabilizing, or losing its coherent self-model.

The authors argue that MetaMo helps design AGI systems that are stable but still adaptable, capable of safe incremental self-improvement, trustworthy in multi-agent settings, and scalable through parallel sub-agents. They use an online research assistant as a running example throughout the paper.

The main point is that no matter how good a system's learning, reasoning, memory, or perception is, if its motivation is broken it can still end up acting stupidly or self-destructively. This is true for humans and equally true for AI. So finding the right motivational framework is really important, but it has gotten relatively little attention because the AI field has historically focused on problem solving and data analysis, which don't usually need complex motivation.

With that in mind, the authors present MetaMo as a general conceptual and mathematical framework for designing and analyzing motivation in complex cognitive systems. The main focus is AGI, but it also applies conceptually to biological intelligence. The framework is built around an abstract categorial model of motivation and how it connects to the rest of an agent's operation, using tools from functional analysis and topology where needed. The sequel paper grounds this abstract model in specific equations by connecting it to OpenPsi and MAGUS. This first paper instead explores the elegant and cognitively relevant abstract properties of MetaMo.

### ***1.1 Brief Comparison to Alternatives***

MetaMo is different from the older ways of doing motivation in AI. The big difference is that MetaMo gives you one solid mathematical foundation, instead of just a pile of tricks and rules patched together.

They compare it with two common older approaches. The first is reward-maximizing agents, which just try to maximize one fixed reward number. The second is hand-tuned drive hierarchies, where a designer manually sets up the agent's drives and the agent switches between them. The problem with both is that they break easily. The reward-maximizer falls apart when the reward situation changes, and the drive hierarchy falls apart when two drives end up conflicting.

MetaMo avoids this because it treats motivation as one combined mathematical object with both a decision side (monad) and an appraisal side (comonad), plus the stability and topology guarantees. So instead of building each property separately by hand, things like modularity, stability, and reachability all come naturally out of the same framework. That is why the agent stays predictable even when it is changing itself.

They also explain how MetaMo relates to OpenPsi. OpenPsi was used in the older OpenCog versions and gives a lot of detailed variables and appraisal algorithms, which you actually need if you want to run real experiments. MetaMo sits one level above that as a more abstract design language. In the sequel paper they show that OpenPsi can be fitted into MetaMo neatly. So MetaMo basically does two things at once: it contains approaches like OpenPsi, and it gives a clear path for upgrading them with proper guarantees about stability, scalability, and cooperation between agents.

## 2 MetaMo: Categorical Core

The starting point is to model the agent's motivational state as an object  $X = G \times M$  inside a category  $C$ . In the sequel paper this gets made concrete:  $G$  is a vector of goal intensities (how strongly the agent wants each goal) and  $M$  is a vector of modulator values like valence, arousal, ethical vigilance, social drive, energy level, and safety margin. So  $X$  bundles together all the drive strengths and all the mood and context signals into one single object.

Then they put two structures on  $X$ . The first is a comonad for appraisal, which updates the agent's feelings and modulators. The second is a monad for decision, which scores and selects goals. These two interact through something called a lax distributive law, which just means that "feel then choose" and "choose then feel" can give slightly different results, but the difference is small and controlled.

A **comonad** is a concept from category theory, but for your purposes here's what matters: in MetaMo it's just the **mathematical label for the appraisal ("feel") step**.

The intuition behind a comonad is "a value that comes with surrounding context, and you can extract from it or extend computations over it." A monad is often described as a way to *build up* or *produce* a value (good for decisions, choosing an output). A comonad is the mirror image: it's good for *reading from* a value that sits in a context and *updating based on that context*. That's exactly what appraisal does. It takes the agent's current state in its environment and reads off an updated mood from it.

So the pairing in MetaMo is deliberate:

- I. **Comonad ( $\Psi$ ) = appraisal = "feel"**  $\rightarrow$  reads context and updates modulators
- II. **Monad ( $D$ ) = decision = "choose"**  $\rightarrow$  produces and selects an action

Because of this, the combined operator  $F = D \circ \Psi$  inherits both the monad and the comonad structure at once, which they call a pseudo-bimonad. The benefit of this is that you can nest appraisal and decision loops inside each other without the system losing coherence. Finally, a motivational coalgebra  $\alpha$  governs one full cycle of appraisal plus decision, so one application of  $\alpha$  is basically one complete "feel and choose" step.

### 2.1 Interpreting Appraisal and Decision in a Perception-Action Loop

At time  $t$ , the agent's motivational state is  $x_t = (g_t, \mu_t)$ , where  $g_t$  are the goal intensities and  $\mu_t$  are the modulators (mood, caution, energy, etc.). The agent perceives an environment state  $s_t$ , which includes both what it observes outside and its internal context, like the memory of what it recently did. Then it runs the appraisal operator  $\Psi$  on its state and what it perceives, giving an updated state  $x'_t = \Psi(x_t, s_t)$ . This is the "feel" step.

**Appraisal** means the agent **sizing up a situation and judging what it means for itself**.

When something happens (a stimulus, a query, an event), the agent evaluates it: is this good or bad, safe or risky, useful or threatening for my goals? That evaluation is the appraisal, and it updates the agent's mood and modulators.

appraisal is the **"feel" step** (the operator  $\Psi$ ). It takes the agent's current state plus what it perceives, and produces an updated emotional/motivational state.

So the simple flow is:

something happens  $\rightarrow$  **appraisal** (judge what it means for me)  $\rightarrow$  mood and modulators update  $\rightarrow$  then decision (choose what to do)

Next, the decision operator  $D$  takes that updated state and picks an action  $a_t = D(x'_t)$ , by scoring each possible action under the now-updated goals and modulators. This is the "choose" step. Executing the action produces a new environment  $s_{t+1}$ , which again includes a record of the action it just took. The full cycle is captured by the coalgebra  $\alpha$ , which combines appraisal and decision into one update, often just written  $x_{t+1} = F(x_t, s_t)$ .

**Modulators** are the agent's internal **tuning dials** that set its current mood and mental state, and that shape how it perceives, thinks, and acts.

continuous internal parameters (like arousal, valence, caution) that represent the agent's mood and adjust how it perceives, thinks, and chooses, separate from its actual goals.

### *The lax distributive law*

is the rule in MetaMo that connects appraisal and decision, and it basically says: **the order you do them in barely matters**.

You have two steps: appraisal  $\Psi$  (feel) and decision  $D$  (choose). You could run them in two orders:

- **feel then choose:**  $D \circ \Psi$
- **choose then feel:**  $\Psi \circ D$

A *strict* distributive law would say these two give exactly the same result. But that is too strong for a real system. So MetaMo uses a **lax** version. "Lax" means "loose" or "approximate." The lax distributive law says the two orders give *almost* the same result, differing only by a small, controlled amount.

Written in the paper as:

$$\lambda_X: \Psi(D(X)) \Rightarrow D(\Psi(X))$$

This is the formal object (a transformation) that maps one ordering to the other and guarantees the gap between them stays small.

**Why it matters.** This small guaranteed gap is what gives the whole system its robustness. Because the order does not really change the outcome:

- You can nest and interleave feel and choose steps freely without the system contradicting itself.
- You can swap out or upgrade the appraisal part or the decision part independently.
- You avoid the human failure mode of "I thought this would make me feel good, but after I did it I felt bad," because predicting-then-choosing and choosing-then-feeling line up.

**The practical condition** (from section 2.1): for this to actually hold, the appraisal operator has to see the same context in both orderings. They achieve this by including the agent's own recent action in what it perceives, so both paths work on the same inputs.

The lax distributive law guarantees that if you swap the order, that is, first predict the post-action modulators for each candidate and then choose, you end up with an action and a next state that differ only by a small, controlled amount from doing it the other way.

There is an important practical condition for this to actually work. The appraisal operator  $\Psi$  has to see the same post-decision context in both orderings. The trick they use is to include the agent's own recent action and updated internal state inside what it perceives ( $s_t$ ). That way the "predict then act" path (appraising using simulated decision info) and the "act then appraise" path (appraising after the action is done) are both working on the same inputs. So their outputs only differ by the small distortion that the lax law already allows.

### 3 Three Categorical Symmetry Principles for Robust AGI Motivation

To explicate the structure and meaning of the category-theoretic setup at the heart of MetaMo, we articulate three "meta-motivational" symmetry principles trivially derived therefrom.

#### 3.1 First Principle: Modular Appraisal-Decision Interface

$$\begin{array}{ccc}
 \Psi(\mathbb{D}(X)) & \xrightarrow{\lambda_X} & \mathbb{D}(\Psi(X)) \\
 \Psi(\mu_X) \downarrow & & \downarrow \mu_{\Psi(X)} \\
 \Psi(X) & \xrightarrow{\lambda_X} & \mathbb{D}(X)
 \end{array}$$

Basic concept: You can swap updating "how you feel" and choosing "what to do" in either order and still end up in essentially the same state.

Appraisal ( $\Psi$ ) and decision ( $\mathbb{D}$ ) each affect a different part of the agent's state. Appraisal changes the feelings (modulators), and decision changes the goal intensities. These are separate channels, so the two steps do not step on each other. The lax law  $\lambda$  guarantees that running appraisal then decision differs only by a small controlled amount from running decision then appraisal.

#### Technical Explanation

It sets up the precise mathematical setting. It uses a structure called a commutative quantale ( $Q$ ), which is basically the system that measures and tracks the "small distortions" mentioned earlier, and it works inside a special kind of category  $C$  that is enriched over  $Q$ . The point of all this machinery is just to give a rigorous home for the idea that things only have to commute "approximately."

In that setting, the appraisal comonad  $\Psi$  and the decision monad  $\mathbb{D}$  each come with extra maps (called unit and tensor 2-cells) that let them combine over multiple states at once, while still obeying the usual coherence rules up to those small  $Q$ -distortions. The tensor maps  $\psi_{X,Y}$  and  $d_{X,Y}$  are what allow appraisal and decision to be applied to combined states, which is what makes the parallel-composition principle possible later.

The lax distributive law  $\lambda$  is then formally defined as a family of these 2-cells that make the four Beck diagrams commute up to the  $Q$ -weights. ("Beck diagrams" are the standard diagrams you check when you want a distributive law between a monad and comonad, and "commute up to  $Q$ -weights" again just means approximately, within the bounded distortion.)

***Distortion*** here just means a **small error or difference** between two results.

In this context: when you do "feel then choose" versus "choose then feel," you don't get the exact same answer. The little gap between the two answers is the distortion. It is how much the result gets "bent" or "thrown off" by doing the steps in a different order.

The whole point of MetaMo is that this distortion stays **small and controlled**, not big and unpredictable. So you can reorder, nest, or combine the steps and the errors only add up a little bit, never enough to break the system.

A simple analogy: imagine translating a sentence English  $\rightarrow$  French  $\rightarrow$  English. You don't get back the exact original sentence, but you get something very close. That tiny difference is the distortion. As long as it stays small, the meaning survives.

***A commutative quantale*** is the mathematical tool MetaMo uses to **measure and keep track of the small distortions** (errors) between results.

Here is the plain idea. Remember the distortions, the little differences between "feel then choose" and "choose then feel." You need some structured way to talk about how big those differences are and what happens when you combine many of them. A commutative quantale is that structure.

Breaking down what it gives you:

- It has a notion of **ordering** ( $\leq$ ), so you can say one distortion is smaller than another. This lets you say "the error is at most this much."
- It has a way to **combine** distortions (the  $\otimes$  operation), so when you stack two steps, you can compute the total error from the two individual errors.
- **Commutative** means the order of combining doesn't matter: distortion A combined with B equals B combined with A. This fits the whole MetaMo theme that order shouldn't matter much.
- It has an **identity element** ( $e$ ), which represents "no distortion at all," the perfect case.

So you can think of a quantale as a generalized "ruler for errors." Normal numbers let you add up errors ( $0.1 + 0.2 = 0.3$ ). A quantale is the more abstract, general version of that idea, built so it works inside category theory.

Why MetaMo needs it: when you nest and interleave many appraisal and decision steps, each one adds a little distortion. The quantale is what lets the authors prove that all those small distortions combine into a still-small total, rather than blowing up. That guaranteed bound is what keeps the system coherent.

## ***Commutative Appraisal-Decision Example***

This is a concrete numerical example showing the lax distributive law actually working. The point is to prove that "feel then choose" and "choose then feel" give the same answer. Let me walk through the numbers.

**The setup.** The current satisfaction level is some value  $x$  in  $[0, 1]$ . There are two candidate actions, A and B, with payoffs  $p_A = 0.6$  and  $p_B = 0.4$ .

Two operators are defined:

- I. Appraisal  $\Psi$  predicts the satisfaction after each action:  $\Psi(x) = (x + p_A, x + p_B)$ . So it adds each action's payoff to the current satisfaction.
- II. Decision D picks the best one:  $D(s_A, s_B) = \max(s_A, s_B)$ . So it just keeps the higher value.

### ***Path 1: appraise then decide ( $D \circ \Psi$ ).***

First appraise:  $\Psi(x) = (x + 0.6, x + 0.4)$ . Then decide:  $D(x + 0.6, x + 0.4) = \max(x + 0.6, x + 0.4) = x + 0.6$ .

### ***Path 2: decide then appraise ( $\Psi \circ D$ , then $\mu$ ).***

First decide:  $D(x) = x + \max(p_A, p_B) = x + 0.6$ . Then appraise that result:  $\Psi(x + 0.6) = (x + 0.6 + p_A, x + 0.6 + p_B) = (x + 1.2, x + 1.0)$ . Then apply the monad multiplication  $\mu$ , which again takes the max:  $\max(x + 1.2, x + 1.0) = x + 1.2$ .

**The conclusion.** Both paths around the square, "appraise then decide" and "decide then appraise," end up at the same updated satisfaction. This shows the lax distributive law in action: in a properly designed motivation system, predicting how you will feel and then choosing gives the same outcome as choosing and then checking how you feel. They commute up to a small controlled distortion.

The real-world point: this is exactly what prevents the human-style mistake of "I thought doing this would make me feel good, but once I did it I felt bad." Because the two orders agree, the agent's predicted feelings and actual feelings line up.

## *Application to Trading Agents*

This is a real-world example of the first principle, using an AI trading agent. The agent researches financial trades and then automatically executes them. Its appraisal operator  $\Psi$  updates modulators like risk-aversion and anticipated regret based on market signals, and its decision operator  $D$  scores and selects which trades to make.

The example shows the two possible orderings:

**Choose then appraise.** The agent first runs  $a = D(x)$  to pick the highest-scoring trade under its current state, executes it, then calls  $\Psi$  afterward to update its modulators based on the actual profit or loss it experienced.

**Appraise then choose.** The agent first runs  $\Psi$  on each candidate trade, predicting how its modulators would look after each trade, then runs  $D$  on those predictions to choose the trade whose imagined outcome looks best.

The lax distributive law guarantees that these two orderings give a trade and updated modulators that are very close together, with the distance between them bounded by a small  $\delta$ :

$$d(D(\Psi(x)), \Psi(D(x))) \leq \delta$$

The benefit is twofold. First, the agent avoids the human-style error of "I thought buying this would make me feel good, but I ended up feeling nervous," because its predicted feelings and actual feelings line up. Second, since appraisal and decision are modular, a team can optimize or swap out either  $\Psi$  or  $D$  independently while still keeping a consistent trading style, even when real market feedback comes in.

### 3.2 Second Principle: Reciprocal Motivational State Simulation

**Basic concept:** If updating agent A and then translating its state into agent B gives the same result as translating first and then updating B, then B can faithfully simulate A.

Imagine two agents, A and B, each with its own "appraise then decide" update cycle. There is a translation map  $T$  that converts A's internal motivational state into B's. The principle says: if running A's update and then translating always gives the same answer as translating first and then running B's update, then B can shadow A step for step. In other words, B can stand in A's shoes and follow along exactly.

#### Technical Explanation

Let  $\alpha: X \rightarrow F(X)$  and  $\alpha': X' \rightarrow F'(X')$  be the one-step coalgebras of agents A and B. These are just the update rules:  $\alpha$  is one full "appraise then decide" cycle for A, and  $\alpha'$  is one cycle for B.  $X$  is A's motivational state space and  $X'$  is B's.

A functor  $T: X \rightarrow X'$  is the translation map. It takes a motivational state from agent A and converts it into the corresponding state in agent B.

A natural transformation  $\varphi_X: T(F(X)) \Rightarrow F'(T(X))$  handles that same translation but at the level of the update operators, lining up "translate A's updated state" with "update B's translated state."

These satisfy, for every map  $f: X \rightarrow Y$ , the **naturality identity**:

$$F'(T(f)) \circ \varphi_X = \varphi_Y \circ T(F(f))$$

This identity is the consistency rule. It says that translating and updating fit together the same way no matter which state you start from. The order in which you apply translation and the internal maps does not change the result.

From this it follows that:

$$\varphi_X \circ T(\alpha) = \alpha' \circ T$$

Read this directly: on the left, run A's update  $\alpha$  and then translate (with  $\varphi \circ T$ ). On the right, translate first ( $T$ ) and then run B's update  $\alpha'$ . The equation says these two are equal. So **T commutes with the updates**, which makes  $T$  a coalgebra homomorphism. In plain words, B simulates A exactly.

$$\begin{array}{ccc}
 X & \xrightarrow{\alpha} & F(X) \\
 T \downarrow & & \downarrow \phi_X \\
 X' & \xrightarrow{\alpha'} & F'(X')
 \end{array}$$

commutes, proving that  $T : (X, \alpha) \rightarrow (X', \alpha')$  is a coalgebra morphism.

**The diagram.** The block of symbols at the top is just a commutation square (the PDF mangled it). It shows the four corners  $X$ ,  $F(X)$ ,  $X'$ ,  $F'(X')$  connected by the maps  $\alpha$ ,  $T$ ,  $\phi$ , and  $\alpha'$ . Saying the diagram "commutes" means both paths around the square give the same result, which proves that  $T$  is a coalgebra morphism. This is the visual version of the equation from the last section: translate-then-update equals update-then-translate. You can redraw it as a clean square in your doc or just describe it.

perfect commutation is impossible in real code. So instead they use a PAC-style  $\phi$ . (PAC means "probably approximately correct," a standard idea from learning theory.) This means  $\phi$  is allowed to be off by at most  $\epsilon$ , with probability  $1 - \delta$ . So  $B$  does not track  $A$  perfectly, but it tracks  $A$  within known, bounded error most of the time.

### ***PAC Learning (Probably Approximately Correct)***

PAC learning is a foundational concept from machine learning theory, introduced by Leslie Valiant in 1984. The name tells you the whole idea: a learner can't be perfect, so the best you can ask for is that it is **probably approximately** correct.

Break the name into its two parts:

**Approximately correct.** The learner doesn't have to get the exact right answer. It just has to get close, within some small error margin called  $\epsilon$  (epsilon). So instead of demanding "zero error," you demand "error at most  $\epsilon$ ." For example,  $\epsilon = 0.05$  means the learner is allowed to be wrong on up to 5% of cases.

**Probably.** The learner doesn't even have to hit that "approximately correct" bar every single time. It only has to hit it **most of the time**, with high probability. That probability is controlled by  $\delta$  (delta). So you demand "the learner is approximately correct with probability at least  $1 - \delta$ ." For example,  $\delta = 0.01$  means it succeeds 99% of the time.

Put together, a PAC guarantee says: **with probability at least  $(1 - \delta)$ , the learner's error is at most  $\epsilon$ .** Two knobs, two kinds of slack.  $\epsilon$  controls how close,  $\delta$  controls how often.

A simple analogy: imagine training someone to recognize spam emails. A perfect filter is impossible. A PAC guarantee would say something like "99% of the time (that is the probably part), this filter will misclassify no more than 5% of emails (that is the approximately part)." You accept a little error and a little chance of failure, in exchange for something achievable.

**Why it matters.** The key insight of PAC learning is that perfect learning from finite data is impossible, so you formalize "good enough" instead. Given enough training examples, you can make both  $\epsilon$  and  $\delta$  as small as you want (tighter accuracy and higher reliability), but you can never drive them to exactly zero. There is always some residual risk and some residual error.

### 3.3 Third Principle: Motivational Compositionality

**Basic concept:** Updating two independent motivational sub-agents in parallel and then merging their states gives essentially the same result as merging them first and then updating.

**What it means.** Imagine splitting your assistant into two modules. One module manages curiosity and exploration, and the other manages user-service and ethics. Each module has its own appraisal-decision loop, so each one feels and chooses on its own.

In a perfect system, these two modules would share no variables at all. That means you could run them side by side at the same time and then just join their results together, with no interference between them. The order would not matter: update-then-merge would equal merge-then-update.

In reality, things are messier. The two modules do share some things, like caches, memory, and safety checks, so once in a while they step on each other and interfere. But the principle says that as long as that interference is small most of the time, you still get a coherent combined update. The little clashes do not break the system.

#### Technical Explanation

So  $X \otimes Y$  means "the state of module X merged with the state of module Y." The "symmetric" part just means merging X with Y is the same as merging Y with X, the order doesn't matter.

The appraisal comonad  $\Psi$  and decision monad  $D$  are called **lax-monoidal** if they come with structural maps that let them work across merged states:

$$\begin{aligned}\psi_{X,Y}: \Psi(X) \otimes \Psi(Y) &\rightarrow \Psi(X \otimes Y) \\ d_{X,Y}: D(X) \otimes D(Y) &\rightarrow D(X \otimes Y)\end{aligned}$$

Read these directly. The first one says: if you appraise X and appraise Y separately and then merge them, you can relate that to appraising the merged state  $X \otimes Y$  directly. The second says the same thing for decision. These maps are exactly what let "do it separately then merge" connect to "merge then do it." They satisfy the usual rules (unit and associativity) up to small coherent distortions ("2-cells").

Then the combined operator  $F = D \circ \Psi$  inherits its own version of this merging map:

$$\varphi_{X,Y} = d_{\Psi(X), \Psi(Y)} \circ D(\psi_{X,Y}): F(X) \otimes F(Y) \rightarrow F(X \otimes Y)$$

This is just the two structural maps chained together, first the appraisal-merge map  $\psi$ , then the decision-merge map  $d$ . The existence of this combined map  $\varphi_{X,Y}$  is the formal proof of the principle: it witnesses that *updating the two modules in parallel and then merging agrees, up to the small coherence distortion  $\varphi$ , with merging them first and then updating.*

## 4 MetaMo: Dynamical and Topological Aspects

the paper is shifting gears from the neat structural math to the messier dynamics-and-stability math. Both are needed, the clean algebra for how the pieces fit together, and the messier analysis for how the system stays stable as it evolves.

### 4.1 Fourth Principle: Homeostatic Motivation Stability

This is the first dynamical principle, and it is about keeping the agent inside a safe, desirable region of its motivation space.

**The basic idea.** There is a desirable region  $R$  inside the full state space  $X$ . For example,  $R$  could be the set of states where goal shifts are gentle and growth stays controlled, basically the "healthy zone." The rule is:

- I. As long as the agent's state stays **inside**  $R$ , it evolves freely with no constraints. It can explore and change however it wants.
- II. But when it gets **close to the edge** of  $R$ , a pulling-back mechanism (boundary-driven contractivity) kicks in and pulls it back inside.

So it is like a fence with a soft spring near the edge. In the middle you roam freely, but near the boundary you get gently pushed back toward the center. This is what "**homeostatic**" means, the system self-corrects to stay in balance.

boundary band  $B_\eta$ , which is the thin strip near the edge of  $R$  (all the states in  $R$  that are within distance  $\eta$  of leaving  $R$ ):

$$B_\eta = \{x \in R : \inf_{z \notin R} d(x, z) \leq \eta\}, \eta > 0$$

Then two requirements:

1.  $F(R) \subseteq R$ , meaning the update keeps states that are in  $R$  still in  $R$  (you don't get kicked out by an update).

2. The contraction rule, but only near the boundary:

$$d(F(x), F(y)) \leq c d(x, y) + \epsilon \text{ whenever } x \in B_\eta \text{ or } y \in B_\eta$$

with  $c < 1$  and  $\epsilon \geq 0$ .

Read this as: when a state is in the boundary band, the update **contracts** distances (since  $c < 1$ , two states get pulled closer together, plus a tiny slack  $\epsilon$ ). This contraction is what prevents the agent from escaping the region. But deep inside  $R$ , away from the boundary, this contraction is **not** imposed, so the agent keeps full flexible dynamics there.

### ***Homeostatic Motivation Stability (qualitative, application, technical)***

**Qualitative summary.** The gist of homeostatic motivation stability: when the agent notices its motivations drifting toward extremes, it automatically soothes them, for example by raising caution or lowering excitement. Then in its decision step it follows those soothed signals to make only small, careful adjustments. This is a two-phase feedback loop. Whenever the agent gets near the edge of its comfort zone, it gently steers itself back toward the safe region.

In terms of cognitive processing, the agent runs **two passes each cycle**:

- I. First, **appraisal ( $\Psi$ )** checks how close the state is to the edge of region  $R$  and raises caution modulators (like safety margin or ethical vigilance) if needed.
- II. Then, **decision ( $D$ )** scores and bumps goal intensities, but those bumps are scaled down by the current modulators.

So when the state nears the boundary, the update pulls nearby states closer together (stopping escape), while deep inside  $R$  it allows more flexible shifts.

### ***Application To Research Assistants***

Picture a fleet of AI research assistants, each maintaining a safe region  $R$ , for example states where goal-intensity changes stay moderate. Each cycle:

- I. The appraisal pass  $\Psi$  checks the current state  $m_t$ . If it is near the edge (in the boundary band  $B_\eta$ ),  $\Psi$  raises caution modulators, like boosting a "stability" signal.
- II. The decision pass  $D$  then scales down any proposed goal-intensity jumps by those modulators, so big jumps get damped whenever the state is near the edge.

The nice consequence: when the state is well inside  $R$ , the caution modulators stay low, so the assistant can chase extreme novelty and curiosity, exploring "crazy" research hypotheses and unorthodox directions. But if that exploration pushes it toward the boundary, caution automatically rises in the next appraisal pass, tempering the novelty and steering it gently back. So you get bursts of creative exploration without the agent ever derailing badly, a balance of innovation and stability.

**Technical version.** Here  $X = G \times M$ , with  $G$  the goal-intensity sliders and  $M$  including a "stability" modulator. They define the safe region as states where the goal change from the previous step is small:

$$R = \{(g, m) \mid \|g - g_{\text{prev}}\| \leq \delta\}, B_\eta = \{x \in R \mid \text{dist}(x, X \setminus R) \leq \eta\}$$

On the boundary band, appraisal and decision each individually contract distances:

$$d(\Psi(x), \Psi(y)) \leq c_1 d(x, y) + \epsilon_1, d(D(u), D(v)) \leq c_2 d(u, v) + \epsilon_2$$

So the combined operator  $F = D \circ \Psi$  obeys  $d(F(x), F(y)) \leq c d(x, y) + \epsilon$  with  $c = c_1 c_2 < 1$ . Since both factors are less than 1, their product is even smaller, so  $F$  contracts even more strongly than either step alone.

Concretely, the mechanism is: when the proposed goal change  $\|\Delta g\|$  approaches the limit  $\delta$ , appraisal  $\Psi$  raises the stability modulator  $s$ , and decision  $D$  scales the proposed change by  $(1-s)$ , which damps the big jump. When  $\|\Delta g\|$  is well below  $\delta$ , then  $s$  is near 0 and  $D$  allows larger changes. So near the boundary  $F$  pulls states back into  $R$ , while deep inside  $R$  the assistant explores freely.

**simple:** the agent has a built-in two-step self-calming loop. Appraisal raises caution when it senses it is near the edge, and decision then makes smaller moves because of that caution. This keeps it stable near the boundary and free in the middle.

## 4.2 Continuity of Self and Growth

MetaMo wants an agent to be able to move from its current motivational state to a desired target state, step by step, but with one important condition: it should keep a coherent, continuously evolving **self-model** the whole time.

### *Encouraging Continuity of Self-Model*

The working hypothesis. For robust open-ended intelligence, the authors want an agent that can adjust its motivations radically over time without shattering its internal self-model. A self-model, for now, just means an internal subsystem that predicts the agent's own behaviors and states in different situations. Keeping that intact during change is called preserving self-continuity. They are honest that this might not hold for every kind of beneficial intelligence, but it seems very natural for AGIs with roughly human-like cognitive architecture.

*The formal setup.* They introduce a map  $H$  that takes a system state and produces the corresponding self-model:

$$H: X \rightarrow M$$

Then they assume a **Lipschitz condition** on  $H$ :

$$d_M(H(x), H(y)) \leq L_H d_X(x, y) \forall x, y \in X$$

In simple words: this says small changes in the state  $x$  cause only proportionally small changes in the self-model  $H(x)$ .  $L_H$  is just a constant that caps how much the self-model can move relative to how much the state moves. So the self-model can't jump wildly when the state shifts a little.

**Combining it with stability.** Now they bring back the boundary contraction rule from the last section: near the boundary,  $d_X(F(x), F(y)) \leq c d_X(x, y) + \epsilon$ . Putting the two together, each update produces a **bounded self-model shift**:

$$d_M(H(x_{t+1}), H(x_t)) \leq L_H(c d_X(x_t, x_{t-1}) + \epsilon)$$

Read this as: how much the self-model moves in one step is bounded by the Lipschitz constant times the (contracted) state change. Because  $c < 1$  and the step sizes stay bounded, the **total drift** of the self-model over all steps stays finite (it does not blow up to infinity).

## ***Conditions Ensuring Incremental Motivational Adjustment***

lists the formal conditions that guarantee an agent can move toward any reasonable goal step by step without breaking its self-model, then gives a simple example dynamic.

**The four conditions.** For the agent to reliably evolve toward a target while staying coherent, MetaMo assumes:

- I. **Uniform tubular path-connectedness.** Any two states  $x$  and  $y$  can be joined by a continuous path  $\gamma$  where every point along the way has a "thick" tube of width  $\eta$  around it that stays inside the valid space. In plain terms: there is always a wide, safe corridor connecting where you are to where you want to go, not a narrow tightrope.
- II. **Boundary-driven contractivity.** On the boundary band  $B_\eta$ , the update  $F_\alpha$  contracts distances:  $d(F_\alpha(u), F_\alpha(v)) \leq c d(u, v) + \epsilon$  with  $c < 1$ . (Same pull-back-near-the-edge rule from before.)
- III. **Self-model Lipschitzness.** The self-model map  $H$  obeys  $d_M(H(x), H(y)) \leq L_H d_X(x, y)$ . (Small state changes cause only small self-model changes, same as the last section.)
- IV. **Local Lipschitz continuity of  $F_\alpha$ .** Every state has a neighborhood where the update is Lipschitz (changes are bounded), and near the boundary that local factor  $L_x < 1$  (locally contracting).

***What these conditions give you*** Under these assumptions, any starting state  $x_0$  and any reachable target  $x^*$  can be connected by a "thick" path that is chopped into small segments (each shorter than  $\eta/2$ ). At each waypoint along the path, the local contraction (less than 1) ensures the update moves the state closer to the next waypoint, and the Lipschitz property of  $H$  keeps the matching self-model drift small. By repeating these small steps, the agent reliably converges to the target  $x^*$  while keeping its self-model coherent the whole way.

**Example dynamic.** A simple update rule that does all this:

$$F_\alpha(x) = (1 - \alpha) x + \alpha \sigma_\tau(x)$$

This is a blending step: each cycle moves a fraction  $\alpha$  of the way toward the target (the  $\sigma_\tau(x)$  term), keeping most of the current state. Because  $H$  is Lipschitz, the self-model shift each step is bounded:

$$d_M(H(x_{t+1}), H(x_t)) \leq L_H d_X(x_{t+1}, x_t)$$

And under the tubular path-connectedness plus boundary contraction, the iterates trace a smooth, contracting corridor into the reachable region I and converge to a fixed point, without ever disrupting the agent's self-model.

### ***Fifth Principle: Incremental Objective Embodiment***

***”The principle. Informally: once the agent picks a goal within its reachable range, it inches toward that goal a little at a time. Because each move blends only a small fraction of the new ideal into its current state, the self-model shifts only slightly each step. And the safe goal region is shaped so that if the agent ever drifts too close to the edge, the next update automatically pulls it back inside. So the agent follows a smooth, steady path into its chosen goal without ever breaking the continuity of how it understands itself.”***

**Example where it works.** A research assistant has two continuous sliders, curiosity and service priority, each adjustable in small increments. Its target is (0.6, 0.4), which sits in a region with a "thick" corridor (a wide safe path). By moving in small 0.02-sized steps, two things happen at once: it converges smoothly to (0.6, 0.4), and because each state change is small, the self-model drift stays bounded. So it keeps coherent introspection the whole way.

**Example where it fails.** Suppose small tweaks to the agent's goals produce erratic, unpredictable user feedback, so small changes in the state lead to large, unpredictable updates. Then neither the path-connectivity nor the boundary contraction holds. The agent cannot find a reliable sequence of small steps, and its self-model experiences jolting discontinuities. This blocks convergence to the goal, the agent can't smoothly get where it wants to go and loses coherence trying.

### **Technical version of success vs failure.**

In the **success case**, there is a reachable region I that is uniformly tubular, and a global contraction update:

$$F_{\alpha}(m) = (1 - \alpha)m + \alpha m^*, 0 < \alpha < 1$$

so  $d_X(F_{\alpha}(m), F_{\alpha}(m')) \leq (1 - \alpha) d_X(m, m')$ . Since  $(1 - \alpha) < 1$ , distances shrink each step, so the state converges to the target  $m^*$ , and because H is Lipschitz, the self-model drift stays small, preserving continuity.

In the **failure case**, noise or constraints break the tubular connectivity or the boundary contraction. There exist states near the boundary where  $d_X(F_\alpha(m), F_\alpha(m')) > c d_X(m, m') + \epsilon$  with  $c \geq 1$  or a large  $\epsilon$ . Without the contraction ( $c < 1$ ) and small  $\epsilon$ , the iterates can oscillate or diverge, so the state never reliably reaches the target, and the self-model can jump abruptly, shattering coherence.

### ***4.3 Adopting Appropriate Formal Properties as Metagoals***

an agent's goal system should be set up so that the desirable region it tries to stay in is defined, partly, by **maintaining the MetaMo properties themselves**. That includes the lax distributive law between appraisal and decision (from the early sections) and the more topological properties about staying in a safe region and keeping a continuous self-model (from this section).

So the key idea is this loop: part of "being in a desirable state" is "being in a state where the MetaMo principles are obeyed." And that even includes having, as a top-level goal, the goal of staying in a desirable state. In other words, the agent doesn't just happen to follow these principles, it actively wants to keep following them. Preserving the properties becomes one of its own goals.

This works because MetaMo defines "goals" very broadly. A goal can be something concrete like "help the user" or "find a battery," but it can just as easily be a **metagoal** like "stay stable," "keep my appraisal and decision modular," or "preserve my self-continuity."

## ***5 Why the Five Principles Are Effective***

**The hypothesis.** Agents whose motivation systems follow MetaMo, even roughly, tend to reach their goals more reliably, flexibly, scalably, and enjoyably, at least for AGI systems with roughly human-like cognitive architecture. The argument strategy is simple: for each principle, point to situations where breaking it clearly causes inefficiency and a bad experience for the agent.

**Modular Appraisal-Decision Interface.** If appraisal and decision are too tightly coupled, small timing differences can cascade into big feedback loops that hurt reliability and leave the agent internally confused. Example: a trading agent that picks a high-risk trade first and only then updates its risk modulators might buy, see the transaction, raise risk alarms, immediately reverse the position, and repeat. This oscillation wastes resources, lowers profit, and shatters the agent's internal confidence model.

**Reciprocal Motivational State Simulation.** Without accurate state-translation between agents, handoffs turn into miscommunication, dropping efficiency and eroding trust. Example: in a virtual software team of AGI coding agents, if one agent cannot simulate how its colleagues view its development plan, you get confusion, delays, broken coordination, and agents losing confidence in each other.

**Parallel Motivational Compositionality.** When parallel motivation subsystems conflict and there is no coherence mechanism, their outputs fight and behavior becomes erratic. Example: a smart thermostat with separate "energy-saving" and "comfort" modules that are not integrated coherently will alternately freeze and overheat a room, lowering satisfaction and leaving the agent stuck between contradictory directives.

**Homeostatic Drive Stability.** If every new input fully overrides the previous drives, even minor disturbances can send the agent's priorities into wild swings, like a kind of "motion sickness" in its own state. Example: a planetary rover that acts on every fresh sensor reading without damping may switch endlessly between mapping, obstacle avoidance, and self-repair, never finishing any task. The authors note the consequences could be far worse in the context of AGI code self-modification.

**Incremental Objective Embodiment.** Trying to reprogram goals in one big leap risks overshoot or collapse, breaking task continuity. Example: a research assistant told to jump from 10% to 90% curiosity all at once might abandon its ongoing summaries, lose its context, crash its planning, and even "forget who it is," instead of smoothly converging on new priorities.

### ***Limits of the MetaMo Perspective***

The main limitation: their arguments about what goes wrong when you ignore the MetaMo principles are much clearer for AGI with human-like cognitive architecture than for AGI or ASI in general. In particular, the dynamical principles (homeostatic stability, incremental embodiment, self-continuity) may not apply well to radically non-human minds. They point out that it is not even clear that every superintelligence (ASI) would have a coherent self-model the way humans do. So the whole "preserve your self-model" idea might just not make sense for some future systems.

Despite that, they still believe MetaMo is an excellent guide for building motivation systems for the first generation of human-level and slightly superhuman AGI. And their hope is that those systems could then go on to help develop even better, more flexible motivational frameworks down the line.

## ***6 Conclusion***

The authors introduced a unified formal framework for AGI motivation built around five abstract principles: modular appraisal vs decision, homeostatic stability, gradual ideal alignment (incremental embodiment), reciprocal simulation, and parallel coherence (compositionality). Together these give a clear design language for building agents that are robust, adaptable, and trustworthy, whether the agent is doing complex tasks, improving itself open-endedly, or working alongside other agents.

They also point to the next step. The companion paper takes these abstract ideas toward real implementation by fitting OpenPsi inside the MetaMo framework and discussing how to integrate MetaMo with the Hyperon AtomSpace and neural modules. The authors expect that combining the clean abstract theory with practical implementation will speed up building AGI systems that are both powerful and able to work well with human values.